My OverTheWire Bandit Journey

Zunaira Arshad, BS Cyber Security, Sir Syed University

Walkthrough & Cheat Sheet: Levels 0 to 33

Introduction

OverTheWire Bandit is a beginner-friendly wargame that teaches Linux and security fundamentals through hands-on challenges. Each level is an account on a remote machine (`bandit.labs.overthewire.org`, port `2220`). You find a hidden password, then use it to SSH into the next level.

Core skills covered

- **Linux navigation:** `ls`, `cd`, `cat`, hidden files, awkward filenames
- **Text processing:** `grep`, `sort`, `uniq`, `strings`, `diff`, `tr`, `base64`, `xxd`
- **File search:** `find` by owner, group, size and permissions
- **Compression and encoding:** `gzip`, `bzip2`, `tar`, hexdumps, base64, ROT13
- **Networking:** `ssh` keys, `nc` (netcat), `openssl s_client`, `nmap`
- **Permissions and SUID:** running setuid binaries as another user
- **Cron jobs:** reading scheduled tasks and abusing scripts that run as another user
- **Restricted shell escapes:** breaking out of `more`/`vim` and an uppercase-only shell with `$0`
- **Git manipulation:** history, branches, tags, and `git add -f` to push an ignored file

---

Level-wise Breakdown

Level 0: Logging in with SSH

Objective: Log in over SSH and read the `readme` file in the home directory.

```
ssh bandit0@bandit.labs.overthewire.org -p 2220   # password: bandit0
ls
cat readme
```

Key concept: `ssh user@host -p PORT` connects to a remote machine. `cd readme` fails because it is a file, not a directory, so `cat` reads it.

Password for Level 1: 6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR

Level 1: A file named -

Objective: The password is in a file called - in the home directory.

```
ssh bandit1@bandit.labs.overthewire.org -p 2220
ls
cat < -
```

Key concept: `cat -` means "read from standard input" and just waits. Input redirection (`cat < -`) feeds the file's contents through stdin, so - is treated as a filename. `cat ./-` also works.

Password for Level 2: PK8fYLZg2hnHSz83plBL1iEPKdD3QToB

Level 2: Spaces and dashes in a filename

Objective: The password is in a file called `--spaces in this filename--`.

```
ssh bandit2@bandit.labs.overthewire.org -p 2220
ls
cat < "--spaces in this filename--"
```

Key concept: Two problems at once:

- **Spaces:** the shell splits arguments on spaces, so quote the name or escape each space with `\ `.
- **Leading `--`:** `cat` would read the name as an option. Fixes: `cat < "..."` (redirection bypasses option parsing) or `cat -- "..."` (`--` means "options end here").

Password for Level 3: 7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME

Level 3: Hidden files

Objective: The password is in a hidden file inside the `inhere` directory.

```
ssh bandit3@bandit.labs.overthewire.org -p 2220
cd inhere
ls            # looks empty
ls -la        # reveals the hidden file
cat ./...Hiding-From-You
```

Key concept: Names starting with `.` are hidden from plain `ls`. `ls -a` or `ls -la` shows them, including `.` (current directory) and `..` (parent). `find` also lists everything.

Password for Level 4: xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq

Level 4: Finding the human-readable file

Objective: The password is in the only human-readable file in `inhere`.

```
ssh bandit4@bandit.labs.overthewire.org -p 2220
cd inhere
file ./-file*
cat ./-file07
```

Key concept: `file` reports each file's type. Nine were `data` (or another binary type) and one was `ASCII text`, the readable one. The `./` prefix stops filenames starting with `-` from being read as options.

Password for Level 5: 6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG

Level 5: Searching by size and readability

Objective: The password is in a file under `inhere` that is human-readable, 1033 bytes in size, and not executable.

```
ssh bandit5@bandit.labs.overthewire.org -p 2220
cd inhere
file */{.,}* | grep "ASCII text" | grep -v ', with very long lines'
du -b -a | grep 1033
cat ./maybehere07/.file2
```

Key concept:

- `*/{.,}*` uses brace expansion so the glob matches both normal and hidden files in every subdirectory.
- `grep -v` drops matching lines to shrink the list.
- `du -b -a` prints all files with sizes in bytes, so `grep 1033` isolates the target.

Cleaner one-liner:

```
find . -type f -size 1033c ! -executable
```

Password for Level 6: pXa26xhMWaC2SvDotA4r9EgZkulOeSBW

Level 6: Find by owner, group and size

Objective: The password is stored somewhere on the server in a file owned by user `bandit7`, group `bandit6`, 33 bytes in size.

```
find / -user bandit7 -group bandit6 -size 33c 2>/dev/null
cat /var/lib/dpkg/info/bandit7.password
```

Key concept: `find /` searches the whole filesystem. `-user`, `-group` and `-size 33c` filter the results (`c` = bytes). `2>/dev/null` hides the flood of "Permission denied" errors.

Password for Level 7: Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3

Level 7: The word "millionth"

Objective: The password is stored in `data.txt` next to the word `millionth`.

```
grep millionth data.txt
```

Key concept: `grep` searches for a pattern in a big file and prints only the matching line.

Password for Level 8: VR1ljMayciFxbnUokuQmJFw6QC9VKtub

Level 8: The only unique line

Objective: The password is the only line in `data.txt` that occurs exactly once.

```
sort data.txt | uniq -u
```

Key concept: `uniq` only compares adjacent lines, so `sort` must come first. `uniq -u` prints only lines that are not repeated.

Password for Level 9: EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl

Level 9: Strings preceded by =

Objective: The password is one of the few human-readable strings in `data.txt`, preceded by several = characters.

```
strings data.txt | grep '=='
```

Key concept: `strings` extracts printable text from a binary file; `grep` narrows it to the line with the = marker.

Password for Level 10: B0s2khmbT9u0geKuOoVGW3JZKhndE3BG

Level 10: Base64

Objective: The password is in `data.txt`, which contains base64 encoded data.

```
base64 -d data.txt
```

Key concept: Base64 is an encoding, not encryption. `base64 -d` reverses it.

Password for Level 11: pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro

Level 11: ROT13

Objective: The password is in `data.txt`, where all lowercase (a-z) and uppercase (A-Z) letters have been rotated by 13 positions.

```
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Key concept: ROT13 is a simple Caesar cipher. `tr` maps each letter to the letter 13 places along, and applying it again decodes it.

Password for Level 12: GROozWPO8QyN0mGrjUkID0WCYkZiQxrN

Level 12: Hexdump and layered compression

Objective: `data.txt` is a hexdump of a file that has been compressed repeatedly. Recover the password.

```
mkdir /tmp/mywork && cp data.txt /tmp/mywork && cd /tmp/mywork
xxd -r data.txt > data
file data                       # identify the compression type
mv data data.gz && gzip -d data.gz
# repeat: file -> rename -> decompress with the matching tool
#   gzip -d file.gz | bzip2 -d file.bz2 | tar -xf file.tar
cat <final_file>
```

Key concept: `xxd -r` reverses a hexdump into binary. Then loop: use `file` to identify the format, rename with the right extension, and decompress with `gzip`, `bzip2` or `tar` until plain text appears. Working in a `mktemp -d` or `/tmp` directory keeps the home directory read-only issues away.

Password for Level 13: qQYQiHOBPR8zR61qxYqX45quvihF2uzk

Level 13: SSH with a private key

Objective: The home directory has an SSH private key that logs into `bandit14`. No password is given.

```
ssh -i sshkey.private bandit14@localhost -p 2220
cat /etc/bandit_pass/bandit14
```

Key concept: SSH can authenticate with a key file (`-i`). Each level's password is stored in `/etc/bandit_pass/<user>`, readable only by that user.

Password for Level 14: aaWecNkG4FhxJQxz07uiwzVP6bJiYS65

Level 14: Send the password to a port

Objective: Submit the current level's password to port 30000 on localhost.

```
cat /etc/bandit_pass/bandit14 | nc localhost 30000
```

Key concept: `nc` (netcat) opens raw TCP connections, so you can talk to a service directly and pipe data into it.

Password for Level 15: pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7

Level 15: SSL/TLS connection

Objective: Submit the current password to port 30001 on localhost using SSL/TLS.

```
openssl s_client -connect localhost:30001 -quiet
# paste the current level's password and press Enter
```

Key concept: `openssl s_client` is like `nc` but wraps the connection in TLS.

Password for Level 16: kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V

Level 16: Port scanning

Objective: Find which port between 31000 and 32000 speaks SSL and returns credentials (not just an echo).

```
nmap -sV -p 31000-32000 localhost
openssl s_client -connect localhost:<PORT> -quiet
# send the current password; an RSA private key is returned
mkdir /tmp/mykey && cd /tmp/mykey
nano key.private                # paste the key
chmod 600 key.private
ssh -i key.private bandit17@localhost -p 2220
```

Key concept: `nmap -sV` finds open ports and identifies services. Of the SSL ports, only one returns a key. SSH refuses private keys with loose permissions, hence `chmod 600`.

Password for Level 17: pWXMAZoxGC8JmDMfmT5MGEsobMM3vnj2

Level 17: Spot the changed line

Objective: Two files, `passwords.old` and `passwords.new`. The password is the only line that changed between them.

```
diff passwords.old passwords.new
```

Key concept: `diff` shows differing lines: `<` is from the first file, `>` from the second. The password is the line from `passwords.new`.

Password for Level 18: OQxXZjELndr90zuhOTDYBEomI0SZITXI

Level 18: A logout-on-login `.bashrc`

Objective: The password is in `readme`, but `.bashrc` logs you out as soon as you connect.

```
ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme
```

Key concept: SSH can run a single command instead of opening a shell, which happens before the interactive `.bashrc` trap gets a chance to log you out.

Password for Level 19: KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI

Level 19: SUID binary

Objective: Use the setuid binary in the home directory to read the password for the next level.

```
ls -l
./bandit20-do cat /etc/bandit_pass/bandit20
```

Key concept: A SUID binary runs with the privileges of its owner (here `bandit20`), not the caller. That lets it read a file you normally cannot.

Password for Level 20: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA

Level 20: SUID client and a listener

Objective: `suconnect` connects to a port you choose, reads one line, and if it matches the current password, returns the next one.

```
echo -n "$(cat /etc/bandit_pass/bandit20)" | nc -l -p 4444 &
./suconnect 4444
```

Key concept: Run your own listener with `nc -l`, send it the current password, and let the SUID program connect to it as the client. `&` puts the listener in the background.

Password for Level 21: bW9kBv5WC3P4yoDyf12LSdGuNz5ka6hY

Level 21: Cron job (1)

Objective: A program runs automatically at regular intervals. Find out what it does.

```
ls /etc/cron.d/
cat /etc/cron.d/cronjob_bandit22
cat /usr/bin/cronjob_bandit22.sh
cat /tmp/<file_named_in_script>
```

Key concept: Cron configs live in `/etc/cron.d/`. The script copies the password into a world-readable file in `/tmp`, so read it there.

Password for Level 22: RYVux2rHEm9tiXHmLFzuR7Vhx6AZQMEz

Level 22: Cron job (2)

Objective: Another cron job, where the output filename is generated dynamically.

```
cat /usr/bin/cronjob_bandit23.sh
echo I am user bandit23 | md5sum | cut -d ' ' -f 1
cat /tmp/<that_hash>
```

Key concept: The script builds the target filename from `md5sum` of `"I am user $myname"`. Running the same command with `bandit23` substituted reproduces the filename.

Password for Level 23: gKXDTAXnIz3OBxiPjRZ2uqutUlPZrBsw

Level 23: Cron job that runs your script

Objective: A cron job executes every script in `/var/spool/bandit24/foo` as `bandit24`, then deletes it.

```
mkdir /tmp/mydir && chmod 777 /tmp/mydir
cat > /tmp/mydir/getpass.sh <<'SCRIPT'
#!/bin/bash
cat /etc/bandit_pass/bandit24 > /tmp/mydir/pass24
SCRIPT
chmod 777 /tmp/mydir/getpass.sh
cp /tmp/mydir/getpass.sh /var/spool/bandit24/foo/
# wait about a minute for cron to run it
cat /tmp/mydir/pass24
```

Key concept: If a privileged cron job runs files from a directory you can write to, you can make it run your code with its privileges. The output directory must be writable by `bandit24`, hence `chmod 777`.

Password for Level 24: hVQMk3lJNsmQ7VF3ubyrNNBom7BOgVXv

Level 24: Brute-forcing a 4-digit PIN

Objective: A daemon on port 30002 wants the current password plus a secret 4-digit PIN (0000 to 9999).

```
for i in {0000..9999}; do
  echo "$(cat /etc/bandit_pass/bandit24) $i"
done | nc localhost 30002
```

Key concept: Only 10,000 combinations, so send them all in one connection. Correct responses are distinguishable from "Wrong!" lines, and you can filter with `grep -v Wrong`.

Password for Level 25: SoHfqMOEqIX2IYKVciZxvgpR9a2Djx4P

Level 25: Escaping a non-standard shell

Objective: `bandit26` does not use `/bin/bash`; the login shell shows a message and exits. A private key for it is in the home directory.

```
ssh -i bandit26.sshkey bandit26@localhost -p 2220
# Shrink the terminal window BEFORE connecting so `more` has to paginate.
# In `more`, press v to open vim, then in vim:
:set shell=/bin/bash
:shell
cat /etc/bandit_pass/bandit26
```

Key concept: The shell runs `more` on a short text file and exits. If the window is too small, `more` pauses, and pressing `v` opens `vim`. From vim you can set the shell and spawn it.

Password for Level 26: jHdv2ELQhT22BkprMNDjybZDAkw1zeBJ

Level 26: Using the SUID helper

Objective: Get a shell as `bandit26` (previous level) and grab the password for `bandit27`.

```
./bandit27-do cat /etc/bandit_pass/bandit27
```

Key concept: Same SUID technique as Level 19, but reached through the shell escape from Level 25.

Password for Level 27: STJLJBRRphMxKB392CT4iOr5CbzPU9ER

Level 27: Cloning a Git repository

Objective: The password is in a Git repository reachable over SSH.

```
mkdir /tmp/g27 && cd /tmp/g27
git clone ssh://bandit27-git@localhost:2220/home/bandit27-git/repo
cd repo
cat README
```

Key concept: `git clone` over SSH with a non-default port uses the full `ssh://` URL syntax.

Password for Level 28: y8Yd2ssKcpHpud7UvOSOxwamRMzIGIeQ

Level 28: Git history

Objective: The password in the latest `README.md` is redacted. Find it in the history.

```
git clone ssh://bandit28-git@localhost:2220/home/bandit28-git/repo
cd repo
git log
git show <commit_hash_before_the_fix>
```

Key concept: Git keeps history. Deleting a secret in a new commit does not remove it from earlier commits, so `git log` and `git show` reveal it.

Password for Level 29: Em7eGtqaMySwNFjCpwzzHhLhospOcdt0

Level 29: Git branches

Objective: The password is not in the default branch.

```
git clone ssh://bandit29-git@localhost:2220/home/bandit29-git/repo
cd repo
git branch -a
git checkout dev
cat README.md
```

Key concept: `git branch -a` lists local and remote branches. The needed data lives on a different branch (`dev`).

Password for Level 30: jq9Dfg2rXsfYsWMgFuKlXhphjdH7USgX

Level 30: Git tags

Objective: The password is hidden in the repository, but not in any branch or commit message.

```
git clone ssh://bandit30-git@localhost:2220/home/bandit30-git/repo
cd repo
git tag
git show secret
```

Key concept: Tags are named pointers, and annotated tags can carry their own content. `git tag` lists them and `git show <tag>` displays one.

Password for Level 31: 82NkymblpGBYmIXG6ZQ8YldBYstHpfUf

Level 31: Pushing a file that is git-ignored

Objective: Push a file `key.txt` containing `May I come in?` to the `master` branch. The remote replies with the password.

```
git clone ssh://bandit31-git@localhost:2220/home/bandit31-git/repo
cd repo
cat README.md
echo "May I come in?" > key.txt
git add -f key.txt          # .gitignore blocks *.txt, so force it
git commit -m "add key"
git push origin master
```

Key concept: `.gitignore` stops `git add` from staging matching files. `git add -f` forces it. The server then validates the push and returns the password.

Password for Level 32: pWuj5jBQ6IgV0NXwiH6g1pXRF8S1YvbT

Level 32: The uppercase shell

Objective: Every command you type is converted to uppercase, so nothing works. Escape it.

```
$0
cat /etc/bandit_pass/bandit33
```

Key concept: `$0` is the name of the current shell. Uppercasing does not change `$0`, so it expands to `sh` and gives a normal shell that is not filtered.

Password for Level 33: u4P2CyPOwPGLe94RdD9Uo2FxFwvnFswM

Level 33: The finish line

Objective: Final level. Log in as `bandit33` and read the message.

```
ssh bandit33@bandit.labs.overthewire.org -p 2220
cat README.md
```

Key concept: There are no further levels here. The `README.md` congratulates you on completing the wargame.

Cheat Sheet

Navigation and files

Task	Command
Connect over SSH on a custom port	`ssh user@host -p PORT`
Run one command over SSH	`ssh user@host -p PORT cat readme`
Login with a private key	`ssh -i keyfile user@host -p PORT`
List files, including hidden	`ls -la`
Read a file named `-`	`cat < -` or `cat ./-`
Read a file starting with `--`	`cat -- "--name"` or `cat < "--name"`
Filenames with spaces	Quote them or escape spaces as `\`
Check file type	`file ./-file*`
Glob including hidden files	`*/{.,}*`

Searching and text processing

Task	Command
Find by owner, group, size	`find / -user u -group g -size 33c 2>/dev/null`
Find by size, not executable	`find . -type f -size 1033c ! -executable`
Keep matching lines	`grep "text"`
Exclude matching lines	`grep -v "text"`
Unique line only	`sort data.txt | uniq -u`
Readable text from binary	`strings file | grep '=='`
Compare two files	`diff old new`
List files with sizes in bytes	`du -b -a`

Encoding and compression

Task	Command
Decode base64	`base64 -d file`
ROT13	`tr 'A-Za-z' 'N-ZA-Mn-za-m'`
Hexdump back to binary	`xxd -r file > out`
Decompress	`gzip -d`, `bzip2 -d`, `tar -xf`

Networking

Task	Command
Send data to a TCP port	`echo data | nc localhost PORT`
Listen on a port	`nc -l -p PORT`
TLS connection	`openssl s_client -connect localhost:PORT -quiet`
Scan ports and services	`nmap -sV -p 31000-32000 localhost`

Privileges, cron and shells

Task	Command
Spot SUID binaries	`ls -l` (look for `s` in permissions)
Run a SUID helper	`./bandit20-do cat /etc/bandit_pass/bandit20`
Inspect cron jobs	`cat /etc/cron.d/<job>` and the script it runs
Fix key permissions	`chmod 600 keyfile`
Escape `more`	Shrink window, press `v` to open vim, `:set shell=/bin/bash`, `:shell`
Escape uppercase shell	`$0`

Git

Task	Command
Clone over SSH with a port	`git clone ssh://user@host:PORT/path/repo`
View history / an old commit	`git log`, `git show <hash>`
List branches	`git branch -a`
List and show tags	`git tag`, `git show <tag>`
Force-add an ignored file	`git add -f file`

Lessons to remember

1. Always use `ls -la`; hidden files are common hiding spots.
2. Prefix suspicious filenames with `./` so they are never parsed as options.
3. Use `file` before `cat` to avoid dumping binary data into the terminal.
4. Combine tools (`find`, `grep`, `sort | uniq`, `strings`) instead of checking things by hand.
5. Secrets deleted from Git are still in the history.
6. SUID binaries and privileged cron jobs are powerful; misconfiguring them is a classic privilege escalation route.

Progress

Levels	Status
0 to 33	Completed and documented above